

SFC-010 - Structural Feasibility Metric

The Underlying Algorithm Behind Confidence Thickness

Sizhe Tan

with chatGPT (OpenAI)

2026-06-26

Repository: <https://github.com/sizhet/Structural-Feasibility-Confidence-SFC>

Abstract

Previous discussions in this repository introduced Structural Feasibility Confidence (SFC), Confidence Thickness, and Puzzle Reasoning as practical phenomena observed in engineering design, AI reasoning, and Function Tunnel analysis.

This article explains the underlying algorithm responsible for these phenomena.

The key idea is that SFC is fundamentally **not a similarity metric**.

Instead, it is a **Structural Feasibility Metric**, evaluating whether a target structure can be reached through valid structural evolution while satisfying semantic coverage, evolution logic, and immutable constraints.

Confidence is therefore interpreted as the accumulated structural evidence supporting feasibility rather than as an isolated probabilistic estimate.

1. Beyond Similarity

Traditional metric learning attempts to answer one question:

How similar are two structures?

SFC answers a fundamentally different question:

Can one structure feasibly evolve into the other?

This distinction changes the optimization objective entirely.

Similarity measures resemblance.

Feasibility measures transformability.

Engineering, software development, manufacturing, scientific discovery, and puzzle solving are all fundamentally feasibility problems rather than similarity problems.

2. Structural Feasibility Differential Tree

Instead of evaluating feasibility using one global calculation, SFC organizes evidence into a hierarchical differential tree.

Each node represents a partial structural feasibility estimate.

Leaf nodes evaluate local structural evidence.

Intermediate nodes aggregate feasibility recursively.

The root node represents the overall Structural Feasibility Confidence.

This hierarchical organization naturally supports incremental refinement while maintaining global consistency.

3. One Tree, Three Validations

Each node derives its feasibility score from three complementary validation mechanisms.

3.1 Concept Coverage Validation

The first validation determines whether all required concepts are properly covered.

Universal Typing and Naming (UTN) provides standardized semantic hierarchies that minimize ambiguity.

Coverage verification therefore becomes a structural traversal problem rather than a purely linguistic comparison.

Human contracts illustrate this process naturally.

As contractual concepts become increasingly explicit, semantic ambiguity decreases while structural completeness increases.

3.2 Evolution Logic Validation

Coverage alone does not guarantee feasibility.

The proposed transformation must also follow valid structural evolution.

Instead of expressing evolution through increasingly complicated IF-ELSE rules, SFC models evolution using Calling Graphs.

Calling Graphs naturally represent dependency relationships, execution ordering, capability composition, and structural evolution trajectories.

Evolution validation therefore becomes Calling Graph consistency verification.

3.3 Immutable Constraint Validation

Real systems always contain components that cannot be modified.

Standard APIs.

Commercial components.

Industrial standards.

Safety regulations.

These immutable structures define hard constraints on feasible evolution.

When multiple interface combinations exist, Bucket Tree of Permutations (BTP) efficiently explores compatible alternatives while preserving structural consistency.

4. Confidence as Structural Evidence Aggregation

The confidence generated by SFC should not be interpreted simply as probability.

Instead, confidence measures the accumulated structural evidence supporting feasible evolution.

Independent feasibility validations continuously reinforce one another.

As additional structural evidence becomes available, the confidence grows thicker rather than merely larger.

Confidence Thickness therefore represents increasing structural support for a feasible transformation.

5. Relation to Function Tunnel Intelligence

Function Tunnel Intelligence seeks feasible evolution paths rather than arbitrary transformations.

SFC provides the evaluation mechanism that estimates how confidently such tunnels can be followed.

Function Tunnel Confidence is therefore one practical application of Structural Feasibility Metric.

6. Relation to Gap Bridging

Gap Bridging can be interpreted as progressive reduction of Feasibility Distance.

Each successful structural improvement reduces uncertainty while increasing accumulated structural evidence.

Confidence therefore increases naturally as feasibility distance decreases.

7. Relation to Universal Typing and Naming

Concept Coverage Validation depends on explicit semantic organization.

Universal Typing and Naming supplies the differential tree required for structural concept verification.

SFC therefore provides an important algorithmic motivation for UTN beyond naming itself.

UTN supplies explicit semantic structure.

SFC evaluates structural feasibility using that structure.

8. Toward Autonomous Contracting

Perhaps the most important implication extends beyond puzzle reasoning.

Future AI and ASI systems will increasingly need to negotiate, cooperate, and coordinate autonomously.

Such coordination requires explicit feasibility verification before agreements can be established.

From this perspective, contracts become machine-readable feasibility proofs rather than purely legal documents.

Structural Feasibility Metric therefore provides one possible algorithmic foundation for future Autonomous Contracting among intelligent systems.

Conclusion

Structural Feasibility Confidence is best understood not as another confidence estimation technique.

Its deeper contribution is the introduction of a Structural Feasibility Metric that explicitly evaluates transformability through hierarchical feasibility aggregation.

By combining differential trees, concept coverage, evolution logic, and immutable constraint validation, SFC provides a unified framework capable of supporting puzzle reasoning, engineering design, Function Tunnel Intelligence, Gap Bridging, Universal Typing and Naming, and future Autonomous Contracting.

Confidence is no longer simply estimated.

It is structurally constructed.